LANDGUARD AI

Software Requirements Specification & Antigravity Implementation Blueprint

Version 1.0 | 15 October 2026 | Hackathon demonstration | SYNTHETIC DATA ONLY

Product vision: A trustworthy, evidence-first land transaction screening workspace that highlights possible conflicting sales without claiming to prove fraud or legal ownership.

Document control

Field	Specification
System	LandGuard AI
Audience	Hackathon judges, demonstration reviewers, authorized operators
Baseline	Existing Colab-trained models and tested FastAPI backend
Status	Requirements and design specification; not an implementation-complete certificate
Deployment	Vercel: React/Vite frontend; Render/Railway: FastAPI + ML backend

1. Scope, goals and non-goals

The application screens synthetic parcel transactions, explains data inconsistencies, visualizes transaction history, and supports human review. The design must emphasize evidence, traceability, and uncertainty over dramatic fraud scores.

- In scope: interactive landing experience, dashboard, transaction explorer, parcel details, AI assessment, evidence explanations, conflict timeline, review workspace, downloadable reports, preferences, and documented API integration.
- Out of scope for this prototype: official land registry connectivity, NID verification, automated legal determinations, live government scraping, real property ownership certification, and training on real citizens' records.
- Every screen displaying a result must include an accessible "Synthetic demo — screening signal, not proof of fraud" notice.

2. Existing verified technical baseline

Asset	Verified baseline
parcels.csv	3,000 synthetic parcels
ownership.csv	3,000 synthetic ownership rows
transactions.csv	3,851 synthetic transactions
mutations.csv	3,851 synthetic mutation rows
review_cases.csv	1,188 synthetic review cases
Random Forest	landguard_rf_baseline_v1.joblib; 3-class assessment
Isolation Forest	landguard_isolation_forest_v1.joblib; supplementary anomaly signal
Backend	backend/app/main.py; FastAPI tests passed in Colab
API endpoints	/health; /dashboard/summary; /transactions; /transactions/{id}; /parcels/{id}/history; /assess/{id}

Existing test results: 8 endpoint checks returned HTTP 200; unknown transaction returned 404; invalid pagination returned 422. This is a smoke-test baseline, not production readiness. Random Forest was evaluated against labels derived from synthetic indicators; perfect scores do not establish independent real-world fraud-detection performance.

3. Users, roles and permissions

Role	Capability	Status
Visitor	View public landing page and synthetic demonstration	Planned
Reviewer	Search, inspect evidence, annotate and resolve cases	New backend work
Admin	Manage reviewers, access, audit logs, settings	New backend work
Demo mode	Read-only synthetic data, no account required	Recommended initial release

Start with a read-only public demo. Require authentication and server-side authorization before implementing persistent case decisions or sensitive administrative actions.

4. Functional requirements

FR-01 Landing / product story

Distinctive land-record-inspired hero, concise value proposition, interactive sample assessment, animated cadastral/parcel visualization, clear demo disclaimer.

FR-02 Dashboard

Show verified counts, scenario breakdown, recent demo activity and status categories. Charts must label their synthetic source and never fabricate trend data.

FR-03 Transaction Explorer

Search transaction ID or parcel ID; debounced search; server-side pagination; sortable accessible table; filters limited to supported API fields or explicitly added endpoints; loading/empty/error states.

FR-04 Parcel Intelligence

Parcel metadata, ownership snapshot, transaction count and dated history; show missing fields as unknown, not zero.

FR-05 AI Assessment

Display final status, RF class, IF anomaly label and decision score, evidence findings, warnings, assessment date, and human review recommendation.

FR-06 Evidence Explanation

Expandable rule-by-rule rationale; supporting transaction/parcel/mutation references; distinguish raw synthetic flags from independently derived checks.

FR-07 Conflict Timeline

Chronological transaction events, buyers/sellers as synthetic aliases, sale area/share, status and overlapping-claim candidates. Future-dated transactions clearly marked pending.

FR-08 Review Workspace

Case queue, reviewer notes, assign/resolve/reopen, status history and immutable audit trail; requires new authenticated backend endpoints and database.

FR-09 Reports

Export an accessible assessment PDF with timestamp, IDs, findings, caveats and model version. Requires new implementation and validation.

FR-10 Settings

Light/dark/system theme, accessibility motion preference, demo explanation; API base URL configured by environment, not editable for arbitrary public users.

FR-11 Search & navigation

Collapsible desktop sidebar, tablet drawer, mobile navigation, keyboard operation, current-route state and responsive layouts.

FR-12 Error handling

Actionable 404, offline, empty, API failure and validation messages; retry without losing relevant user context.

5. Model and decision requirements

- MR-01: Load the existing joblib artifacts only from trusted local deployment assets; never retrain, overwrite or silently replace them.
- MR-02: Preserve the Random Forest feature order and classes and the Isolation Forest pipeline, features and stored training medians.
- MR-03: Isolation Forest ANOMALY is supplementary; never classify a transaction as fraud solely from anomaly output.
- MR-04: Status precedence: invalid/future transaction date -> PENDING_DATE_VERIFICATION; else RF or evidence REVIEW_REQUIRED -> REVIEW_REQUIRED; else insufficient -> INSUFFICIENT_EVIDENCE; else NO_REVIEW_FLAG_DETECTED.
- MR-05: Multiple sales are review candidates, not conclusive double-selling. Legitimate resale and fractional ownership require human verification.
- MR-06: Preserve and display disagreement between RF and evidence engine, especially when RF says no flag and rules say review.
- MR-07: Record model version, assessment timestamp, synthetic-data provenance and rule version in report metadata.
- MR-08: Existing ownership data lacks a full transfer chain; describe seller checks as snapshot-based, not authoritative title verification.

6. UX concept and visual direction

Design concept: “Cadastral Intelligence Console”. Build visual identity from land survey grids, registry index lines, parcel boundaries, ledger typography and forensic document review. The first memorable visual is an interactive parcel-grid assessment, not a generic gradient card dashboard.

Token	Suggested value	Use
Ink	#172C32	Primary text / deep navigation
Paper	#F7F8F4	Light canvas
Survey teal	#157C78	Primary actions and selected states
Evidence amber	#B66D23	Review caution

Record blue	#426B85	Information and charts
Alert red	#B74445	Error/review only

Typography: Manrope or Plus Jakarta Sans for UI; IBM Plex Mono sparingly for parcel IDs and evidence metadata. Prefer a disciplined 8px rhythm, crisp 6–10px corners, meaningful separators and compact information-dense tables. Avoid generic purple gradients, nested cards, or decorative floating orbs.

7. Required motion and supplied components

- UX-01 Scroll-driven SVG connector: animate an SVG path draw based on scroll progress between the "Record → Compare → Flag → Review" sections; clamp progress; support reduced motion; do not rely on animation to convey information.
- UX-02 MagnificationDock: integrate the supplied React/Framer Motion component as a desktop quick-action dock for Dashboard, Search, Parcel History, Review and Settings. Adapt the code for React/TypeScript compatibility, semantic buttons, keyboard activation, tooltips and responsive fallback.
- UX-03 Prefer purposeful motion: one orchestrated landing reveal, functional expand/collapse and timeline highlighting. Avoid constant dashboard animations.
- UX-04 Optional React Three Fiber parcel terrain/grid visualization only on landing or an opt-in view, lazy-loaded with 2D fallback. Do not block dashboard performance.
- UX-05 Respect prefers-reduced-motion, provide visible focus states and minimum practical 44px touch targets.

8. Responsive layout and design system

Viewport	Navigation	Content
Mobile 320–767px	Bottom primary navigation + accessible drawer	Single-column panels; scrollable table with stable columns
Tablet 768–1023px	Collapsible/drawer sidebar	Two-column summary; readable chart/table regions
Desktop 1024px+	Persistent collapsible sidebar; optional MagnificationDock	Dense workspace, contextual detail panel, four summary metrics

Create semantic design tokens (background, surface, raised surface, text, border, focus, success, warning, error), three-state theme (light/dark/system), real component style guide, accessible toasts, skeletons with stable dimensions, empty states, and typed reusable controls. WCAG 2.1 AA is the acceptance target.

9. Frontend technology and architecture

Vite + React + TypeScript strict + Tailwind CSS + React Router + TanStack Query + Lucide React + Recharts + Motion/Framer Motion. Use Zod for API response validation where useful; React Three Fiber is optional. Implement one typed API client using VITE_API_URL. Do not put credentials or sensitive configuration in Vite environment variables.

Layer	Suggested folders
Routes	frontend/src/pages
Components	frontend/src/components/{layout,ui,assessment,parcel,charts}
Data/API	frontend/src/lib/api.ts and frontend/src/features/*
Design tokens	frontend/src/styles/tokens.css
Types	frontend/src/types
Tests	frontend/src/**/*.test.tsx and e2e/

10. Backend architecture and API contracts

Retain Python/FastAPI and existing ML artifacts. Refactor into app/api, app/services, app/models, app/core, and app/repositories only after preserving current tests. Prefer versioned /api/v1 routes while supporting legacy routes until the frontend migrates.

Method / path	Behavior	Status
GET /health	Model and dataset readiness	Existing
GET /dashboard/summary	Synthetic totals and scenario counts	Existing
GET /transactions?search=&limit=&offset=	Paginated transaction list	Existing
GET /transactions/{id}	Single transaction	Existing
GET /parcels/{id}/history	Parcel and chronological transactions	Existing
GET /assess/{id}?as_of_date=YYYY-MM-DD	Unified RF + IF + evidence response	Existing
GET /review-cases	Case queue with filters	New
PATCH /review-cases/{id}	Authorized case decision and note	New
GET /reports/{transaction_id}.pdf	Generated report	New
POST /auth/login; POST /auth/logout	Secure reviewer session	New

Use consistent JSON errors, Pydantic validation, rate limits, strict CORS allowlist, configurable deployment settings and request logging without personal information. Existing /assess is GET and read-only; do not expose model loading or arbitrary file paths through user input.

11. Database and persistence

Phase 1: read-only CSV and model assets; no database needed for basic demo. Phase 2: PostgreSQL or SQLite for review cases, reviewer accounts, notes and audit events. Never rewrite source CSVs as a substitute for audited case decisions.

Entity	Key fields
review_cases	id, transaction_id, state, assigned_reviewer_id, created_at, updated_at
review_events	id, case_id, actor_id, action, before_state, after_state, timestamp
review_notes	id, case_id, author_id, body, created_at
users	id, email, password_hash or external identity, role, active

All mutable review actions must be transactional, role-authorized and auditable. Use migrations and indexes for transaction_id, case state and timestamps. Node/Express/PostgreSQL example skills supplied by the user are conceptual references, not instructions to replace the working Python backend.

12. Security, privacy and ethical requirements

- SEC-01: Only synthetic demonstration records; do not collect real NIDs, deeds, signatures, phone numbers or private land documents.
- SEC-02: Do not commit secrets, .env, access tokens, private datasets or model artifacts to a public GitHub repository by default.
- SEC-03: CORS restrict to the deployed frontend origin; never rely on CORS as authentication.
- SEC-04: Reviewer/admin actions require authenticated sessions and role checks. Use secure HttpOnly cookies if session authentication is implemented.

- SEC-05: Add request rate limiting, bounded pagination, input validation, safe error responses and dependency pinning.
- SEC-06: Joblib/pickle artifacts are executable on load; accept only trusted local model files and verify provenance/checksums.
- SEC-07: Audit log edits and access to review decisions; redact identifiers and sensitive values from application logs.
- SEC-08: PDF reports must include disclaimer and source provenance, not assert criminal guilt or certified ownership.

13. Quality, performance and accessibility

- NFR-01: Frontend usable from 320px to 1920px with no unintended horizontal page overflow.
- NFR-02: Keyboard navigable; semantic HTML; visible focus; screen-reader labels; WCAG 2.1 AA contrast and reduced-motion support.
- NFR-03: Stable loading skeletons, explicit empty/error/retry states, no fake charts or hardcoded production-like metrics.
- NFR-04: Lazy-load optional 3D and report modules; optimize initial bundle and avoid expensive animations in dense tables.
- NFR-05: Tests for decision precedence, date boundaries, repeated-sale false positives, missing mutation, unknown IDs and API failures.
- NFR-06: Playwright viewport screenshots and keyboard checks, React Testing Library component tests, pytest FastAPI tests.
- NFR-07: Accessibility and responsiveness take priority over visual flourish.

14. GitHub, CI and deployment

Vercel is the selected frontend host for the Vite + React + TypeScript application. Deploy FastAPI separately on Render/Railway (recommended for this existing ML backend), because a Vercel React deployment does not run the Python service or its .joblib models. Connect the two HTTPS services through VITE_API_URL and allow only the actual Vercel origin in FastAPI CORS. The frontend build must use Vite base "/" for a Vercel root-domain deployment.

- Repository hygiene: README, LICENSE, .gitignore, .env.example, frontend/, backend/, docs/, tests/; avoid committing secrets and unreviewed model binaries.
- CI checks: frontend typecheck, lint, build and tests; backend pytest, dependency scan and smoke tests.
- Git workflow: feature branches or staged commits (audit, UI foundation, integration, security, deployment); push only after checks pass.
- README: purpose, architecture, synthetic disclaimer, model limitations, prerequisites, local startup, environment configuration, API docs, test commands, deployment URLs, demo walkthrough and license.
- Antigravity may generate commands and modify local files, but actual GitHub authentication, repository permissions and deployment success must be confirmed from command output; never claim a push or deployment without evidence.

15. Acceptance criteria

ID	Acceptance test
AC-01	Existing backend endpoints retain current behavior and smoke tests pass.
AC-02	All six existing endpoints are called through a typed React API client.
AC-03	Search, parcel history and assessment use real synthetic API responses, not fabricated hardcoded records.
AC-04	TX-002911-1 insufficient; TX-002911-2 review; TX-002911-3 pending for 2026-10-15.
AC-05	Anomaly and RF/evidence disagreement are independently visible.
AC-06	Every assessment carries synthetic disclaimer and

	actionable evidence details.
AC-07	Mobile/desktop navigation, MagnificationDock fallback and scroll SVG pass keyboard/reduced-motion checks.
AC-08	Review mutation and PDF endpoints are either implemented and tested or clearly marked unavailable.
AC-09	Frontend deployed on Vercel; FastAPI deployed independently over HTTPS and health-tested.
AC-10	README, .env.example, tests and deployment instructions committed without secrets.

16. Antigravity staged implementation prompts

Stage 1 — Audit and architecture

Inspect backend, identify model bundle shape and endpoint contracts; create docs/BACKEND_AUDIT.md; run existing tests; do not retrain or overwrite models.

Stage 2 — Design system and UI shell

Create visual tokens, responsive navigation, accessible MagnificationDock, scroll-based SVG journey, and real React app routes; no fake data.

Stage 3 — API integration

Connect summary, transactions, parcel history and assessment to existing FastAPI; add error/loading/empty states and contract tests.

Stage 4 — Review, reports and auth

Implement new persistent review workflow, PDF reporting and authentication only after backend API specification approval; migrate schema safely.

Stage 5 — QA and deployment

Run typecheck, tests, accessibility checks and visual QA; deploy the React frontend to Vercel, deploy FastAPI separately to Render/Railway, and verify both live URLs and cross-origin requests. Update README and CI checks.

17. Open decisions and implementation risks

- Choose whether review workspace remains read-only for the first hackathon demo or requires authenticated persistence.
- Decide backend hosting and model artifact distribution before connecting the Vercel frontend; confirm host memory, startup time, persistence and HTTPS availability.
- Confirm the supplied MagnificationDock source compiles against the selected Motion version; fix keyboard and focus behavior instead of copying unchanged.
- Treat the current as_of_date default (2026-10-15) as a demonstration snapshot; provide an explicit date selector or runtime default with documented semantics.
- Resolve the difference between snapshot ownership data and legal transfer chains; never label repeated sale as proven fraud.
- Confirm whether repository is public before adding synthetic data/model artifacts, and verify their license and provenance.

18. Vercel Deployment Architecture — Approved Update

Deployment amendment | Version 1.1 | 15 October 2026

18.1 Decision and boundaries

Host the React + TypeScript (Vite) frontend on Vercel. Keep the existing FastAPI + Random Forest + Isolation Forest + evidence verification service on a separate Python-capable host such as Render or Railway. Vercel hosting solves frontend deployment, not backend hosting. The two services communicate via HTTPS JSON REST APIs.

GitHub remains the source-control system. Vercel connects to the GitHub repository for preview and production deployments. Backend deploys from the same repository to its own service, with a separate root directory.

18.2 Target deployment topology

- User browser → https://<vercel-project>.vercel.app (React SPA)
- React client → VITE_API_URL=https://<fastapi-service> (HTTPS REST)
- FastAPI → backend/data/*.csv and backend/models/*.joblib (server-side only)
- GitHub repository → Vercel frontend build + separate Render/Railway backend build

18.3 Repository and environment configuration

Repository layout: frontend/ (Vite React application), backend/ (FastAPI app, data, models), docs/ (SRS and audit), tests/, README.md, .gitignore, .env.example. Never commit secrets or personal land records. Decide whether synthetic CSVs and model binaries may be published; otherwise distribute via controlled deployment artifacts.

Vercel project: Framework = Vite; Root Directory = frontend; Build Command = npm run build; Output Directory = dist; install dependencies from lockfile. Set VITE_API_URL to the deployed backend HTTPS origin (no trailing slash). Redeploy after changing a Vite environment variable: VITE_* variables are embedded at build time and are public, so never store secrets in them.

For React Router browser history, add frontend/vercel.json with a rewrite to /index.html, so refreshing /dashboard or /transactions does not return 404. Do not rewrite /assets/* to the SPA when a real asset exists; test deep links. Use base: "/" in vite.config.ts for root-domain deployment.

18.4 Example configuration

frontend/.env.example
VITE_API_URL=https://your-fastapi-service.example.com

frontend/vercel.json
{"rewrites": [{"source": "/(.*)", "destination": "/index.html"}]}

Frontend fetch contract
GET ${import.meta.env.VITE_API_URL}/health; GET /dashboard/summary; GET /transactions; GET /transactions/{id}; GET /parcels/{id}/history; GET /assess/{id}

18.5 Backend deployment requirements

- Use a Python-compatible hosting platform; set backend/ as the service root. Install dependencies from requirements.txt. Start command: uvicorn app.main:app --host 0.0.0.0 --port $PORT (or the host-provided equivalent).
- Ship approved synthetic datasets and trusted joblib artifacts to the backend only. Joblib deserialization is unsafe for untrusted files; do not expose model upload endpoints.

- Configure CORS with the exact production Vercel origin and any explicitly authorized preview origins; avoid wildcard origins when using credentials.
- Expose GET /health and /docs as appropriate; check memory, startup time, timeouts and filesystem availability. A cold start may load both models and CSVs.
- Keep assessment read-only until authenticated review actions are implemented. No real land record, NID or sensitive identity data in the demo.

18.6 Deployment and verification procedure

1. Run backend pytest and smoke tests locally; verify model bundle keys, 404/422 handling and assessment precedence.
2. Run frontend npm ci, npm run build, typecheck/lint/tests; ensure the frontend uses the configured API URL rather than a Colab proxy.
3. Push tested changes to GitHub using meaningful commits; connect the frontend/ root directory to Vercel and deploy.
4. Deploy backend/ separately; verify public GET /health returns HTTP 200 before connecting the frontend.
5. Set VITE_API_URL in Vercel production settings and redeploy; verify dashboard, transaction search, parcel history and assessment over HTTPS.
6. Refresh a nested frontend route, test mobile/desktop, keyboard, error/empty/loading states and reduced-motion behavior.
7. Confirm API CORS headers for the deployed frontend origin and inspect browser Network/Console for failures; never claim deployment successful until live checks pass.

18.7 GitHub automation, README and acceptance criteria

GitHub Actions CI should test frontend and backend on each pull request. Vercel Git integration handles frontend preview/production deploys; optional GitHub Actions may perform smoke checks after deployment. Maintain README instructions for both local and deployed environments, .env.example, API docs URL, test commands, architecture diagram, model limitations and synthetic-only disclaimer.

Acceptance: (1) Vercel production URL loads the app; (2) nested-route refresh works; (3) frontend calls the independently deployed HTTPS FastAPI service; (4) health and assessment tests pass; (5) no secret is bundled into client JS; (6) only synthetic demo data is served; (7) anomaly or review signals are never represented as legal proof of fraud.

18.8 Antigravity implementation instruction — deployment stage

Preserve the trained models and existing working API. Configure Vercel exclusively for frontend/. Add correct SPA rewrites and VITE_API_URL integration. Prepare a separate Render/Railway deployment configuration for backend/. Implement strict origin-based CORS, safe configuration, build/test checks and deployment documentation. Run tests before commits; do not claim a GitHub push or live deployment without terminal and HTTP verification.